Контейнеризация: Docker и Docker Compose

Арзамасова Анна
Эксперт курса «Интеграция ИИ»



Проверка связи

Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти

Поставьте в чат:

\+ если меня видно и слышно

— если нет

Рекомендации

→ Если смотрите с компьютера

- Используйте браузеры Google Chrome или Microsoft Edge
- Если есть проблемы с изображением или звуком, обновите страницу — **F5**

→ Если смотрите с мобильного телефона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение на телефоне
- Предварительно проверьте, подходит ли ваше устройство для подключения к вебинару, по ссылке

Анна Арзамасова

О спикере:

- инженер в сфере авиации, сотрудник НИИ
- специалист по интеллектуальным системам управления транспортом
- преподаватель Московского Политеха
- Преподаватель и разработчик образовательных программ на стыке инженерии, программирования и искусственного интеллекта



Разбор ДЗ вебинара 10

ОРГМОМЕНТ

От стратегии — к технической базе деплоя

Прежде чем переключать трафик между версиями (вебинар 10), нужно, чтобы каждую версию можно было надёжно и одинаково запустить — где угодно.



Формат сегодня

Docker — необязательный дополнительный маршрут для сдачи практики. Основной результат — корректно написанные Dockerfile и docker-compose.yml.

Цели занятия

После занятия вы сможете:

- **Объяснять**, зачем ML-сервису контейнер и чем контейнер отличается от виртуальной машины;
- **Составлять** Dockerfile для FastAPI-сервиса с правильным порядком инструкций;
- **Описывать** в docker-compose.yml связку из API и базы данных для логов;
- **Проверять** Dockerfile по чек-листу: slim-образ, порядок COPY, отсутствие лишних файлов.

План вебинара

1. Зачем нужна контейнеризация
2. Структура Dockerfile для ML-сервиса
3. Docker Compose
4. Kubernetes и облачные сервисы (обзор)
5. Практика: Dockerfile для FastAPI-сервиса

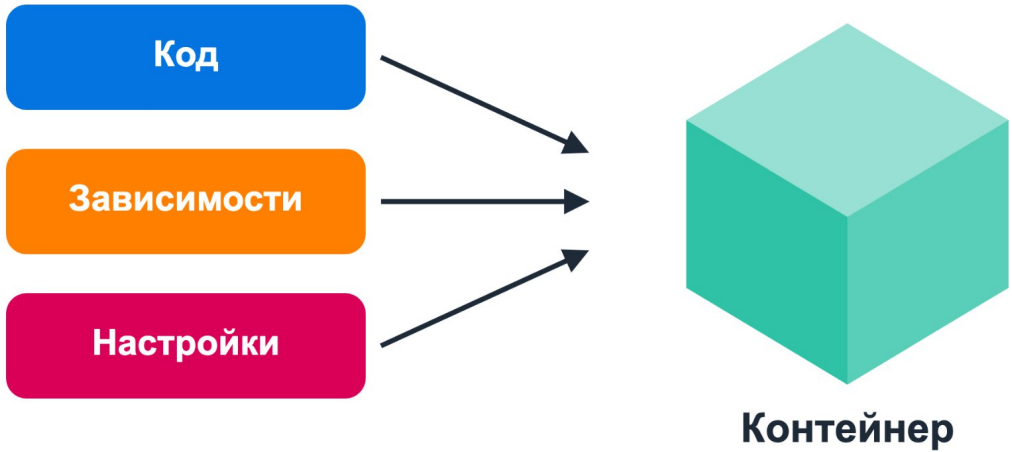
Зачем нужна контейнеризация

ЧАСТЬ 1

«У меня работает, а на сервере — нет» — код зависит не только от себя, но и от окружения

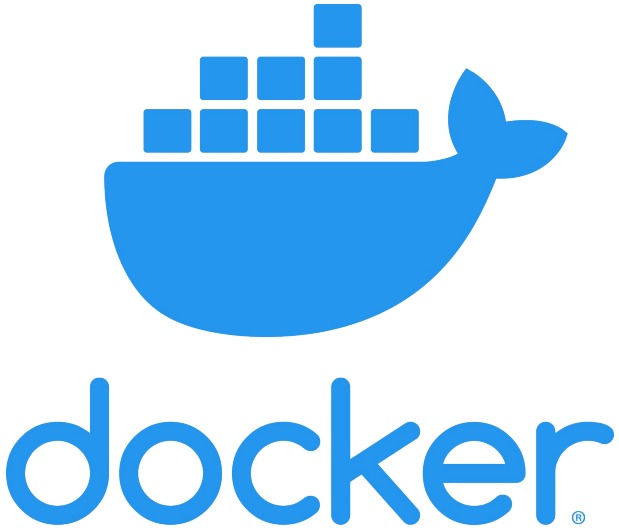
Контейнеризация

Способ упаковать приложение вместе со всем нужным для его работы (код, зависимости, библиотеки) в единый переносимый пакет — контейнер.



Docker

Самый распространённый инструмент для создания и запуска контейнеров.



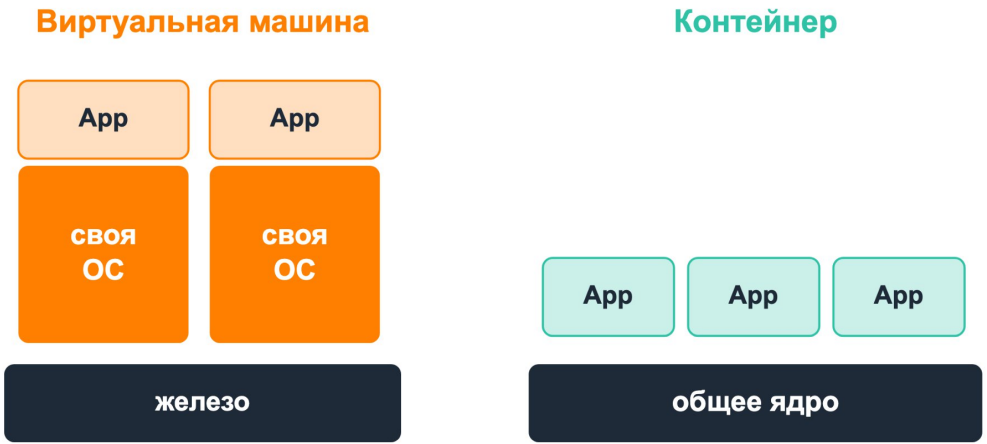
Контейнер vs виртуальная машина

Виртуальная машина

- Эмулирует полный компьютер
- Своя ОС, ядро, драйверы
- Гигабайты, запуск — минуты

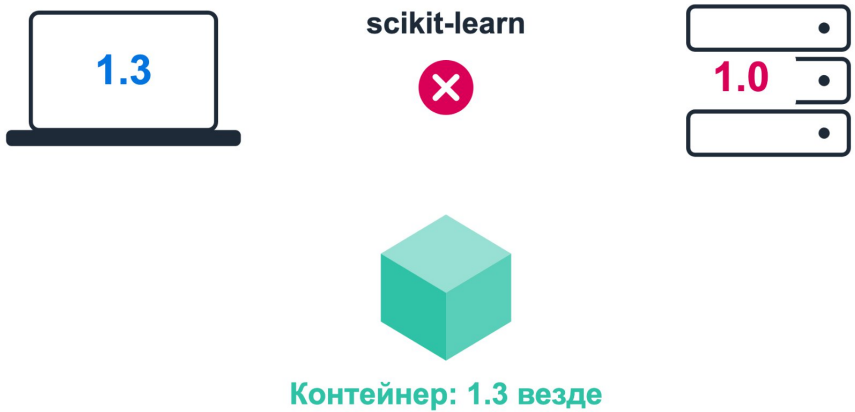
Контейнер

- Использует ядро хост-машины
- Изолирует только процессы и файлы
- Мегабайты, запуск — секунды



Пример

Модель обучена локально с scikit-learn 1.3. На сервере — версия 1.0. Загрузка падает или ведёт себя иначе. Контейнер с зафиксированными версиями исключает эту проблему.

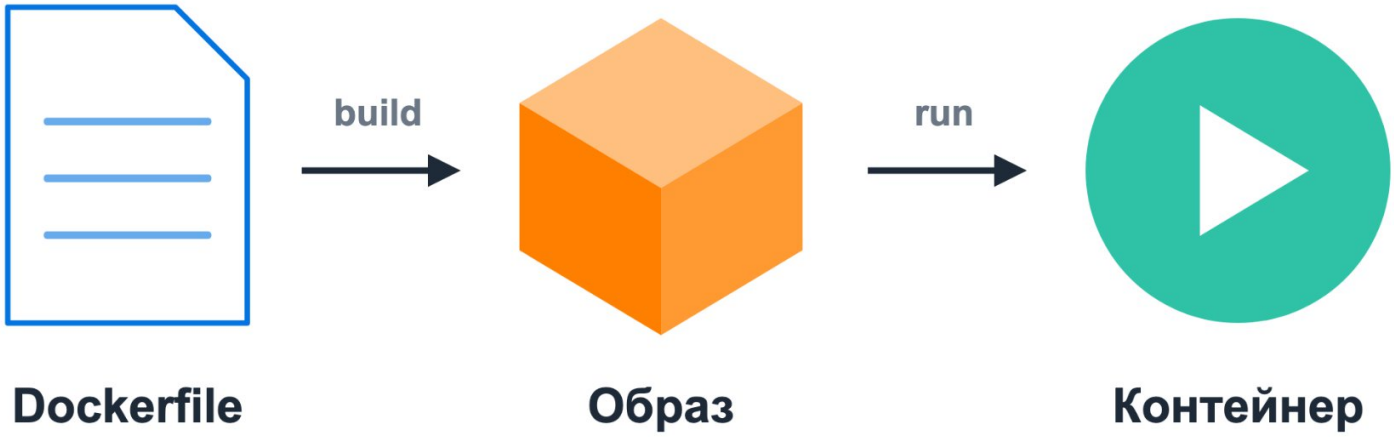


Структура Dockerfile для ML-сервиса

ЧАСТЬ 2

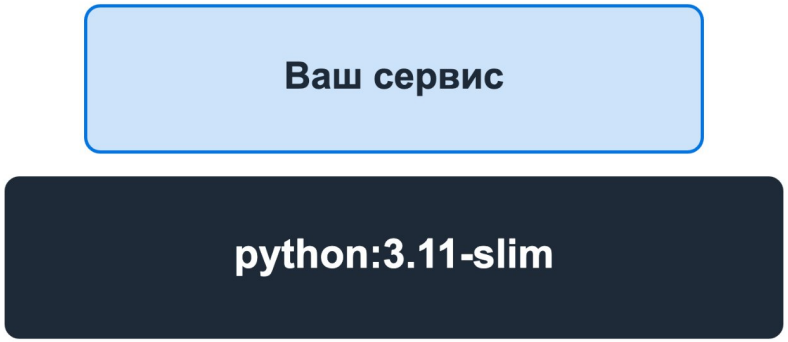
Dockerfile, образ, контейнер

Dockerfile — инструкция сборки. Образ — готовый неизменяемый снимок окружения и кода. Контейнер — запущенный экземпляр образа.



Base image

С чего начинается сборка: минимальный образ нужной версии языка, например python:3.11-slim — облегчённая версия без лишних утилит.



базовый образ — фундамент

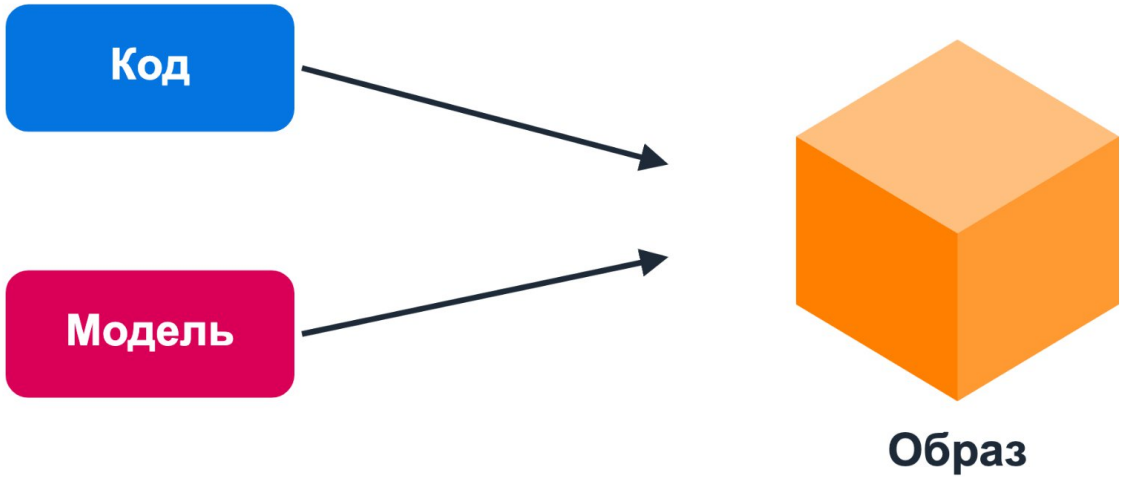
COPY requirements.txt + установка

Копируем список зависимостей и ставим их раньше кода — Docker кэширует шаги, и если зависимости не менялись, шаг переиспользуется из кэша.



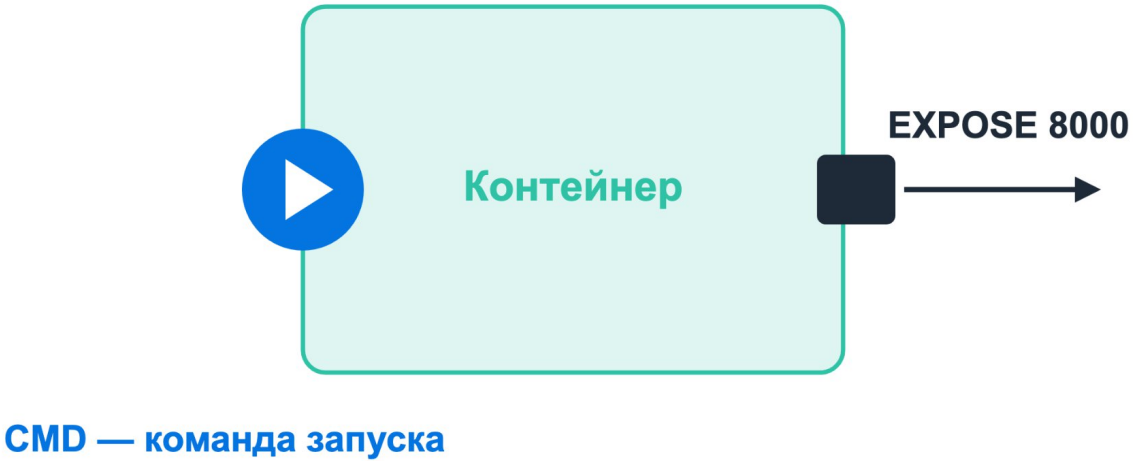
COPY кода и модели

Копирование файлов приложения и сохранённого артефакта модели внутрь образа.



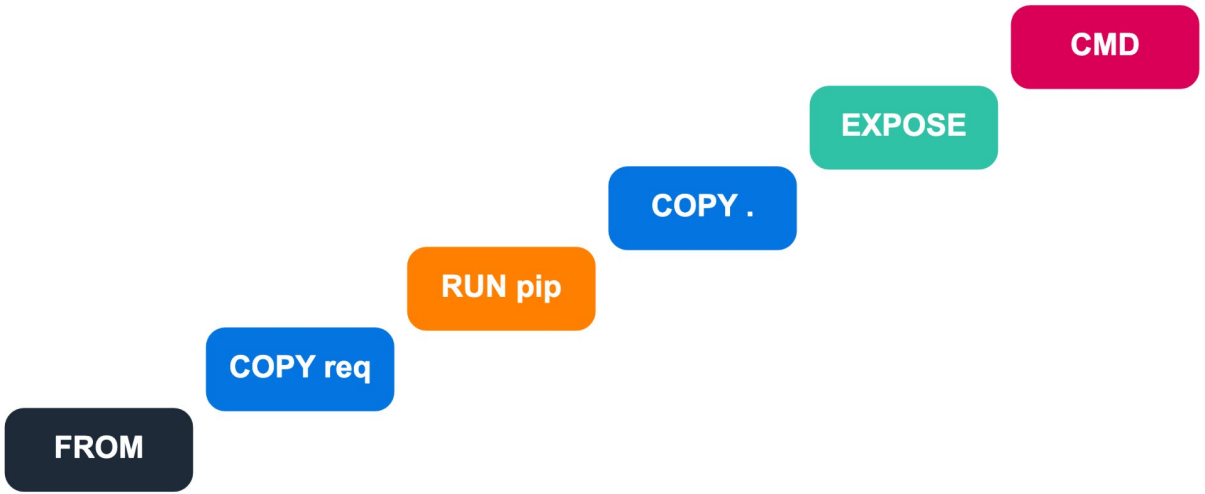
EXPOSE и CMD

EXPOSE — какой порт использует приложение внутри контейнера. CMD — команда запуска, например uvicorn при старте контейнера.



Пример структуры Dockerfile

FROM python:3.11-slim → COPY requirements.txt → RUN pip install → COPY . . → EXPOSE 8000 → CMD uvicorn main:app



Раздельное копирование requirements.txt и кода экономит минуты на каждой пересборке за счёт кэширования

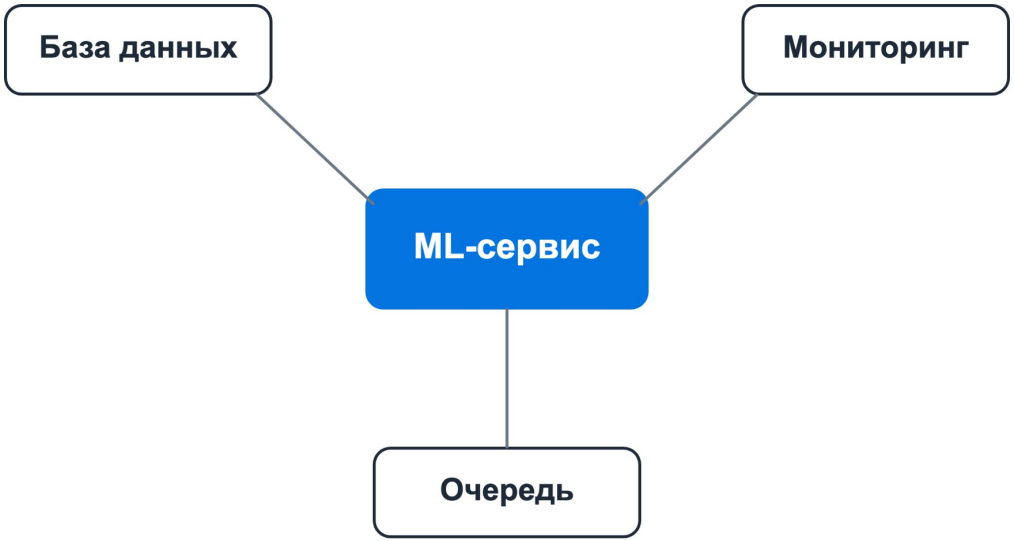
Docker Compose

ЧАСТЬ 3

ML-сервис редко работает в одиночку: ему нужна база данных для логов, иногда — мониторинг, иногда — очередь сообщений

Docker Compose

Инструмент для описания и одновременного запуска нескольких связанных контейнеров через один конфигурационный файл.

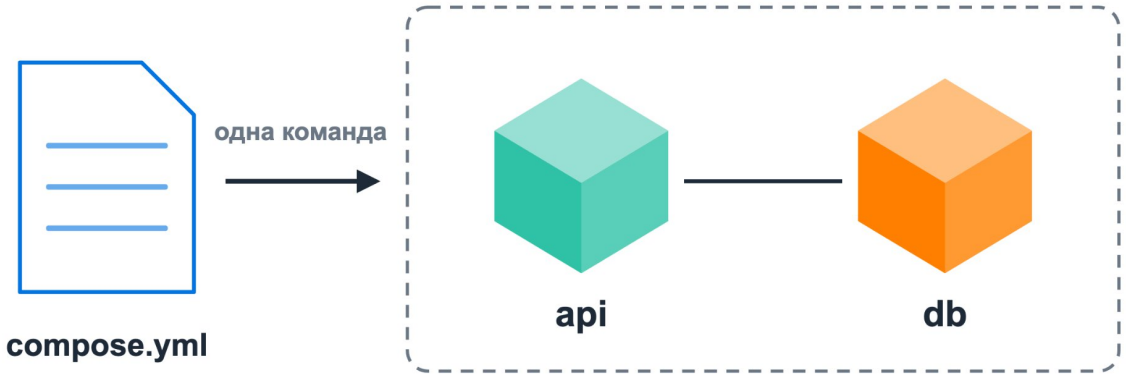


Структура docker-compose.yml

Для каждого сервиса: откуда брать образ, какие порты пробросить, какие переменные окружения, зависимость от старта другого сервиса.

Пример состава

api (FastAPI-сервис из собственного Dockerfile) + db (готовый образ PostgreSQL/SQLite для логов) — запускаются одной командой, видят друг друга по имени сервиса.



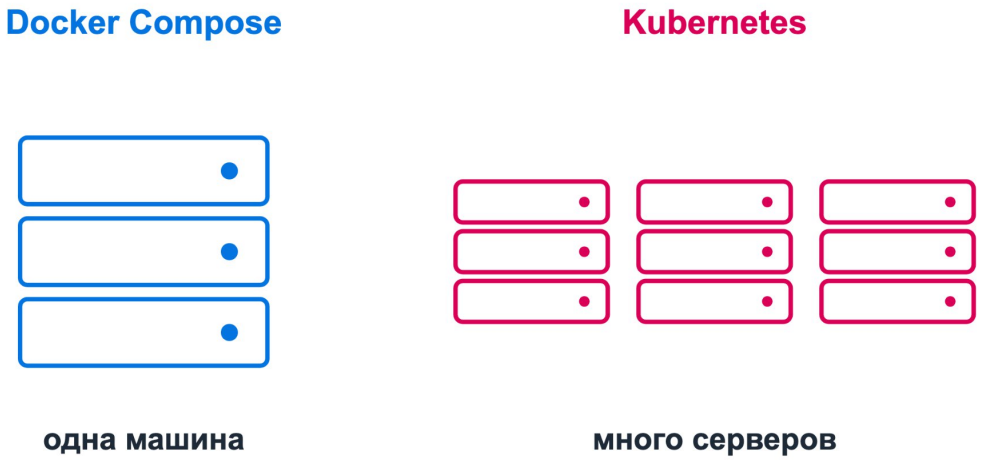
Одна команда поднимает полностью рабочую копию системы со всеми её частями — без ручной настройки

Kubernetes и облачные сервисы

ЧАСТЬ 4

За пределами Docker Compose (обзор)

Kubernetes — система оркестрации контейнеров: распределяет их по серверам, перезапускает упавшие, масштабирует по нагрузке. Облачные managed-платформы берут часть этой работы «под ключ».



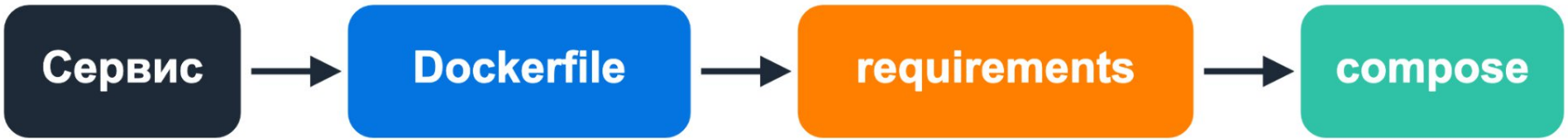
Docker Compose решает задачу воспроизводимости на одной машине. Kubernetes — задачу масштаба на множестве серверов

Dockerfile и docker-compose.yml для FastAPI-сервиса

ПРАКТИКА

Шаг 1

Берём FastAPI-сервис из практики вебинара 9 (три эндпоинта: /predict, /healthcheck, /model-info).



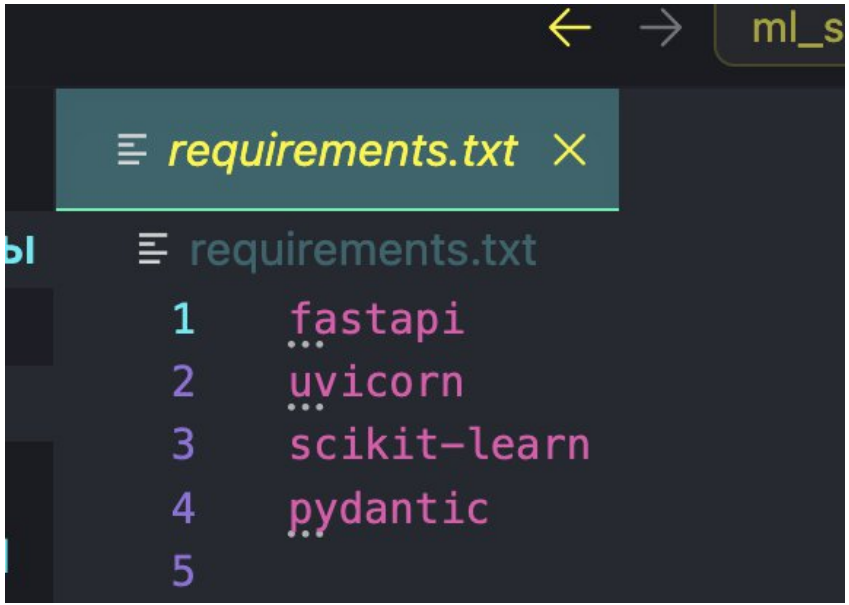
Шаг 2

Пишем Dockerfile пошагово: base image → requirements.txt + установка → код и модель → EXPOSE → CMD.



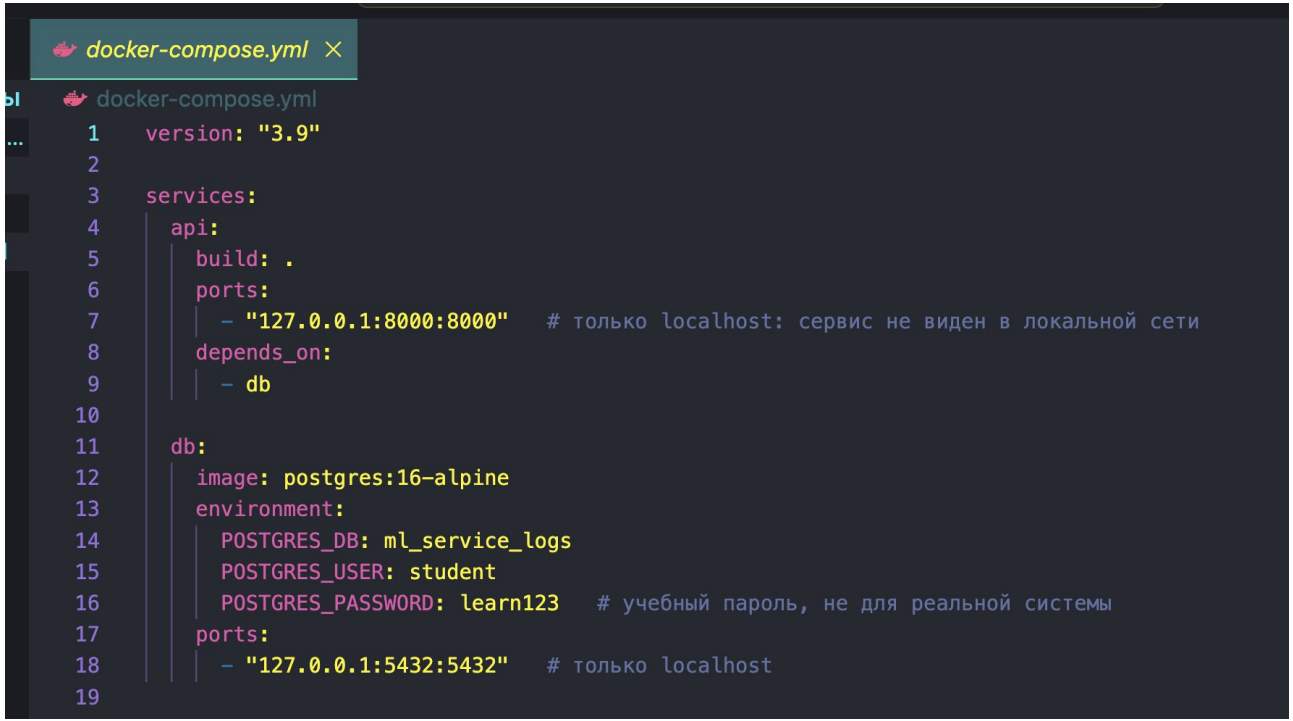
Шаг 3

Разбираем requirements.txt — минимальный набор зависимостей сервиса.



Шаг 4

Пишем docker-compose.yml с двумя сервисами: api и db (SQLite/PostgreSQL для логов).



Чек-лист оптимизации Dockerfile

Используется ли slim-образ

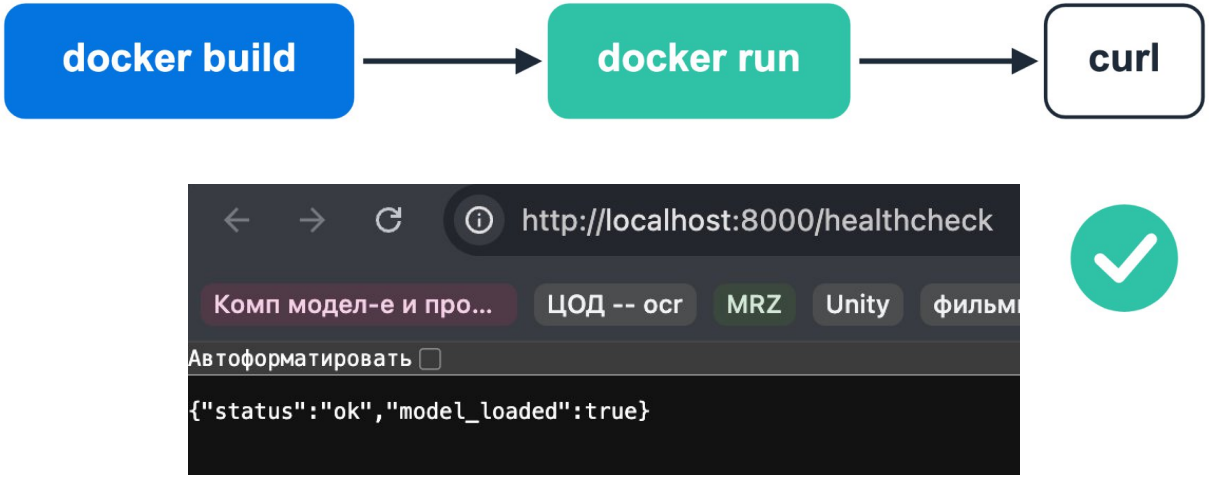
Правильный ли порядок COPY-команд для кэширования

Не копируются ли лишние файлы окружения разработчика

- slim-образ
- порядок COPY
- без лишних файлов

Шаг 6 (опционально, если есть Docker)

Соберите образ (docker build) и запустите контейнер (docker run), проверьте работу через curl или браузер.



Если не работает

Нет установленного Docker
это ожидаемо, достаточно корректно написанных файлов без реальной сборки

Ошибка про отсутствующий файл при сборке
проверьте, что запускаете команду из папки с Dockerfile

Контейнер не отвечает
проверьте совпадение порта в EXPOSE/CMD и в docker run -p



Вы узнали

Главная мысль:

Контейнер упаковывает код и модель вместе с окружением, поэтому сервис одинаково запускается на любой машине, а Docker Compose делает воспроизводимым запуск всей системы из нескольких сервисов.

На практике: *копируйте requirements.txt и ставьте зависимости до копирования кода — тогда при правке кода зависимости берутся из кэша и пересборка идёт быстрее.*

Что мы разобрали сегодня

ИТОГИ

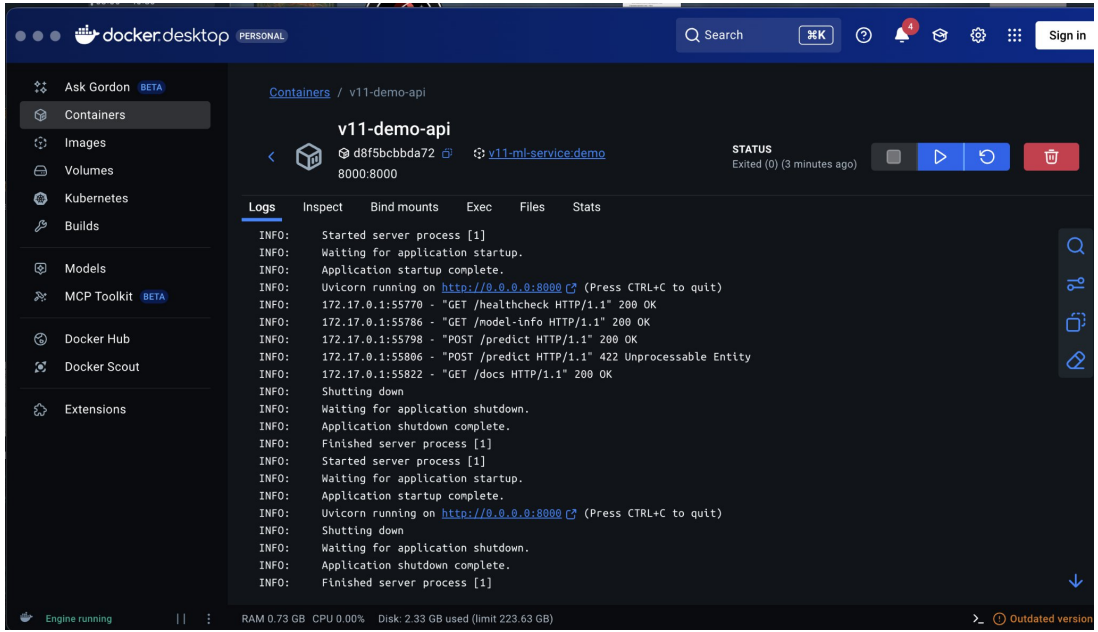
Контейнеризация решает проблему «работает у меня, но не на сервере» — упаковывает приложение вместе с окружением

Dockerfile — воспроизводимый рецепт сборки; порядок инструкций влияет на корректность и скорость пересборки

Docker Compose расширяет контейнеризацию на связанную группу сервисов; Kubernetes — на масштаб множества серверов

Домашнее задание

Создайте Dockerfile и docker-compose.yml для своего проекта; при наличии Docker — проверьте сборку, иначе сдайте корректные файлы с описанием проверки.



Мост к вебинару 12

Последняя тема модуля 3 — мониторинг и поддержка в продакшене: метрики, дашборды, алерты, чтобы узнавать о проблемах раньше пользователей.

СПАСИБО

До встречи на вебинаре 12

Контейнеризация: Docker и Docker Compose

Арзамасова Анна
Эксперт курса «Интеграция ИИ»

